

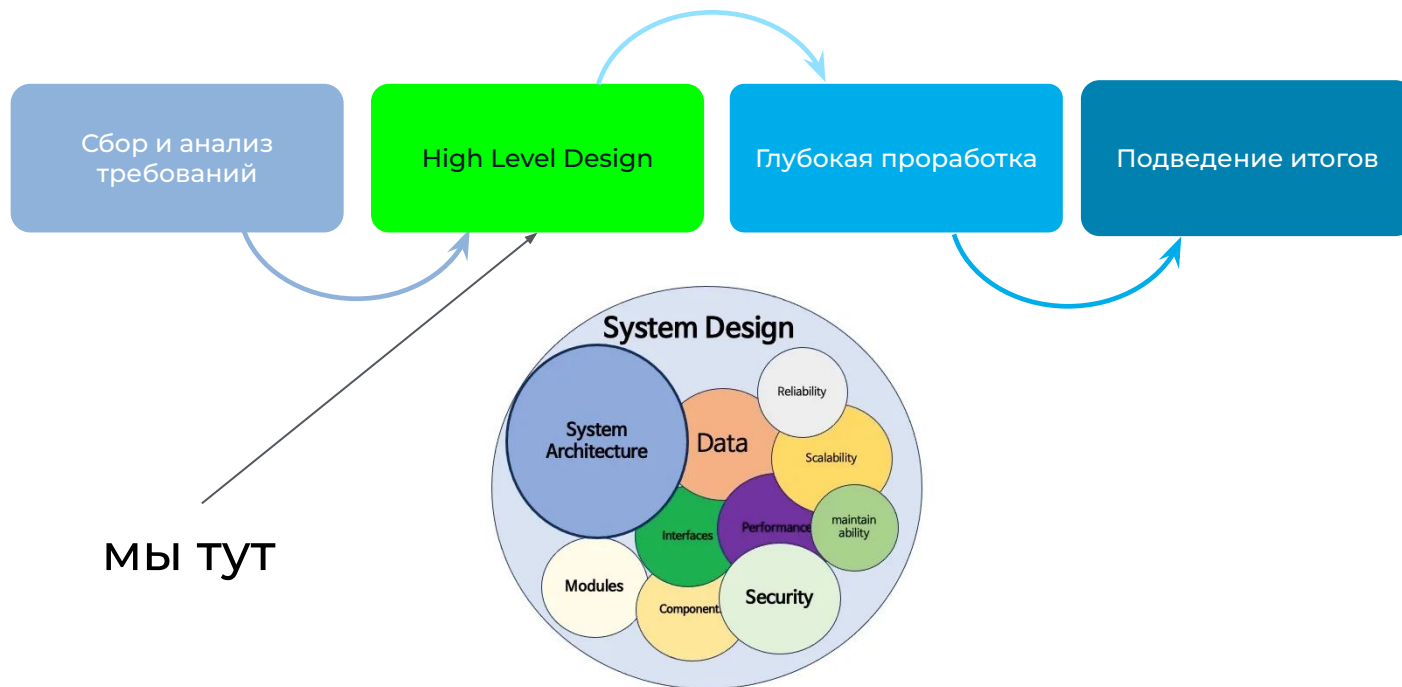


НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ

Системный дизайн современных приложений

Занятие №3
Архитектура. Часть №1

Этапность

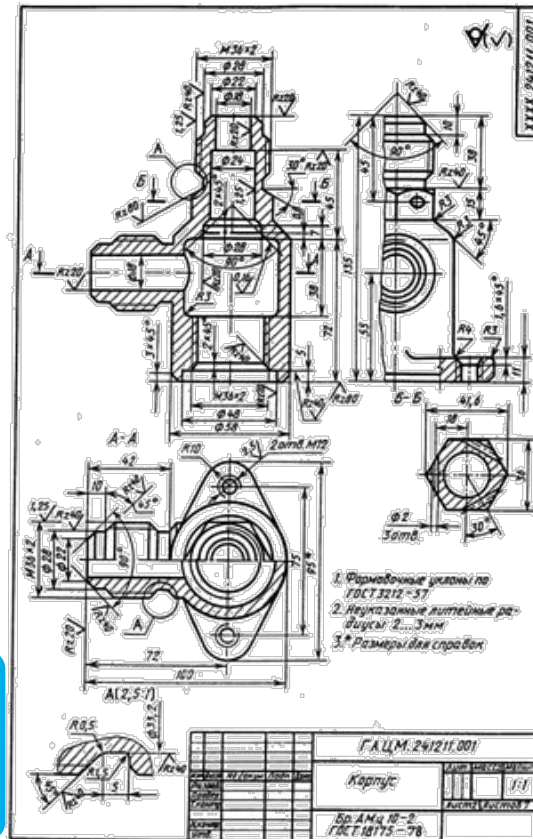


HLD vs LLD

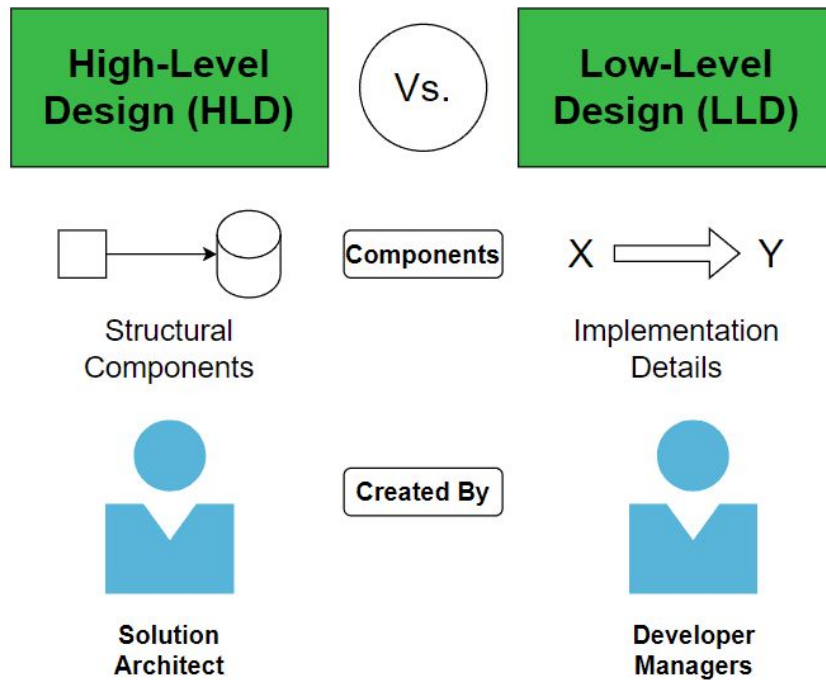


Архитектор

Инженер



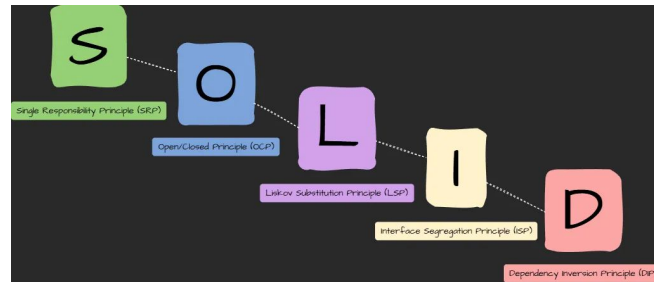
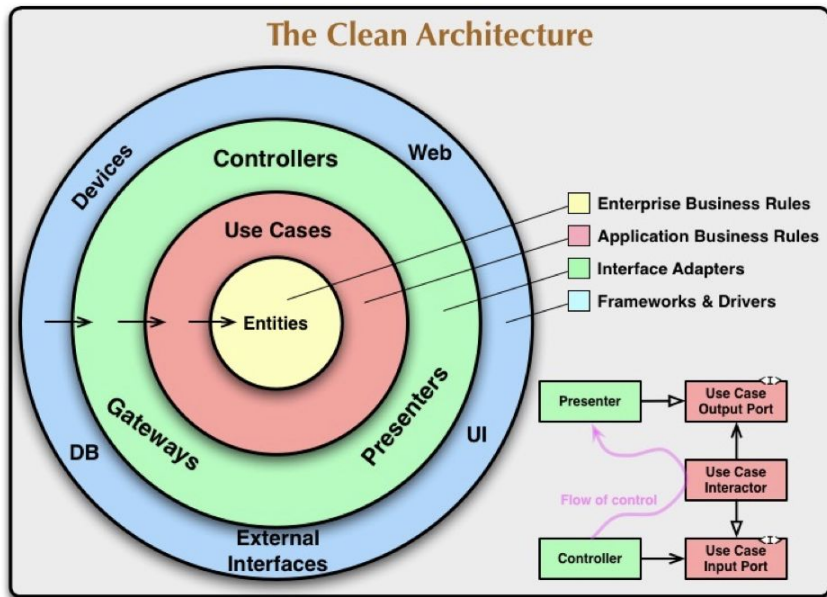
HLD vs LLD



А что вообще такое архитектура?

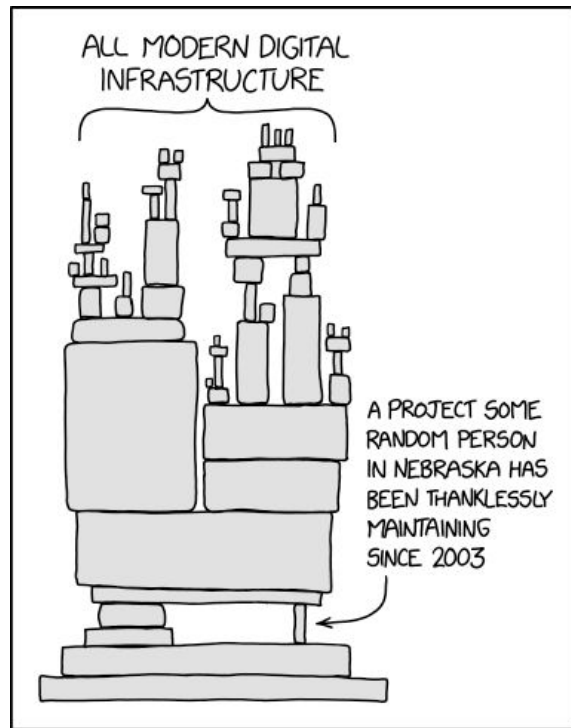
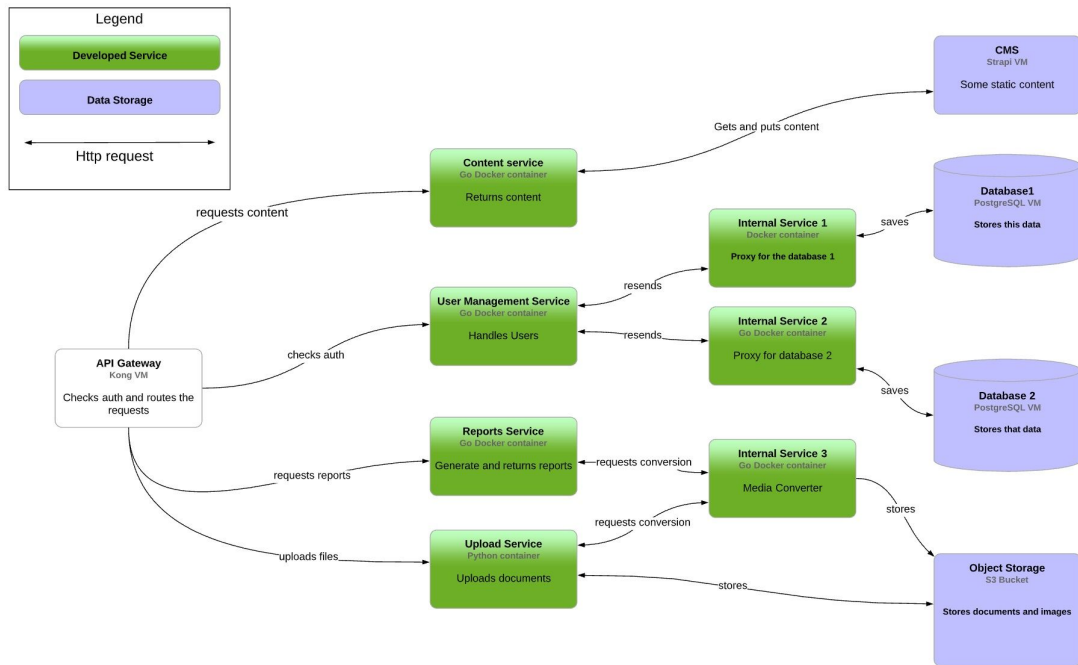


А что вообще такое архитектура?





А что вообще такое архитектура?



Архитекторы

АРХИТЕКТОР РЕШЕНИЙ

Архетип персонажа: детально ориентирован на конкретную систему (например, современная CRM или миграция в облако).



- **Охват:** Отдельный проект/решение.
- **Фокус:** Технические детали, выбор технологий, план реализации.
- **Ключевые результаты:** Детальная проектная документация, диаграммы интеграции, выбор технологического стека.
- **Цель:** Предоставить конкретное, работающее техническое решение.



КОРПОРАТИВНЫЙ АРХИТЕКТОР

Архетип персонажа: стратегические, перспективные системы во всей компании (облако, наследие, данные, безопасность).



- **Охват:** Все предприятие (вся организация).
- **Фокус:** Согласование ИТ-стратегии, межведомственные стандарты, долгосрочные планы, управление портфелем.
- **Ключевые результаты:** Фреймворки ЕА (например, TOGAF), технологические стандарты, карты возможностей, планы
- **Цель:** Оптимизировать общий ИТ-ландшафт организации.

Архитекторы согласуются со стандартами предприятия



Перевод бизнеса для определения потребностей

БИЗНЕС-АРХИТЕКТОР

Архетип персонажа: сотрудничает с бизнес-стейкхолдерами, составляет карту бизнес-возможностей и диаграмму потоков создания ценности.



- **Охват:** Бизнес-возможности, потоки создания ценности, стратегия.
- **Фокус:** Перевод бизнес-целей в структурированный вид, выявление разрывов в возможностях, оптимизация процессов.
- **Ключевые результаты:** Модели бизнес-возможностей, карты потоков создания ценности, потоки бизнес-процессов (BPM).
- **Цель:** Определить и оптимизировать бизнес-структуры и процессы для достижения успеха.



КЛЮЧЕВЫЕ РАЗЛИЧИЯ С ОДНОГО ВЗГЛЯДА

Горизонт времени	Краткосрочный/Проектный	1-3 года	Долгосрочный
Охват	Системный	Общекорпоративный	Бизнес-стратегия
Key Metrics	Успех проекта	Снижение затрат/управление	Реализация бизнес-ценности

- + Один баг вызывает каскадный отказ
- + Система разваливается от изменений
- + Каждая следующая фича дороже предыдущей
- + Давайте перепишем с нуля
- + Сложность при поиске проблем
- + Система задыхается от нагрузки
- + Риски информационной безопасности
- + Растет когнитивная сложность системы
- + Избыточное потребление ресурсов
- + Невозможность гарантировать SLA
- + Высокий порог входа и выгорание





- + Один баг вызывает каскадный отказ
- + Система разваливается от изменений
- + Каждая следующая фича дороже предыдущей
- + Давайте перепишем с нуля
- + Сложность при поиске проблем
- + Система задыхается от нагрузки
- + Риски информационной безопасности
- + Растет когнитивная сложность системы
- + Избыточное потребление ресурсов
- + Невозможность гарантировать SLA
- + Высокий порог входа и выгорание





Архитектор-всезнайка



Архитектор

Инженер (исключен из
процесса)





Участники архитектурного проектирования

Пользователь

Ограничения
Требования
Пожелания

Разработка / QA / Design

Опыт
Технологии
Компетенции

Иные архитекторы

Ограничения
Шаблоны
Тех. радар

Devops / SRE

Инфраструктура
Сопровождение

Бизнес

Бюджет
Требования

Регуляторы / Юристы

Ограничения



Выводы

Архитектура - тот же Системный дизайн

Или можно говорить, что Системный дизайн это процесс, а Архитектура это итог.

Рассматриваем организацию системы

- Компоненты (обработчики, хранилища)
- Связи между компонентами (интеграции)
- Работу системы с внешним миром (клиенты)
- Принципы, определяющие проектирование, разработку, развитие и эксплуатацию системы И компромиссы между ними (атрибуты качества)

Многоэтапный процесс

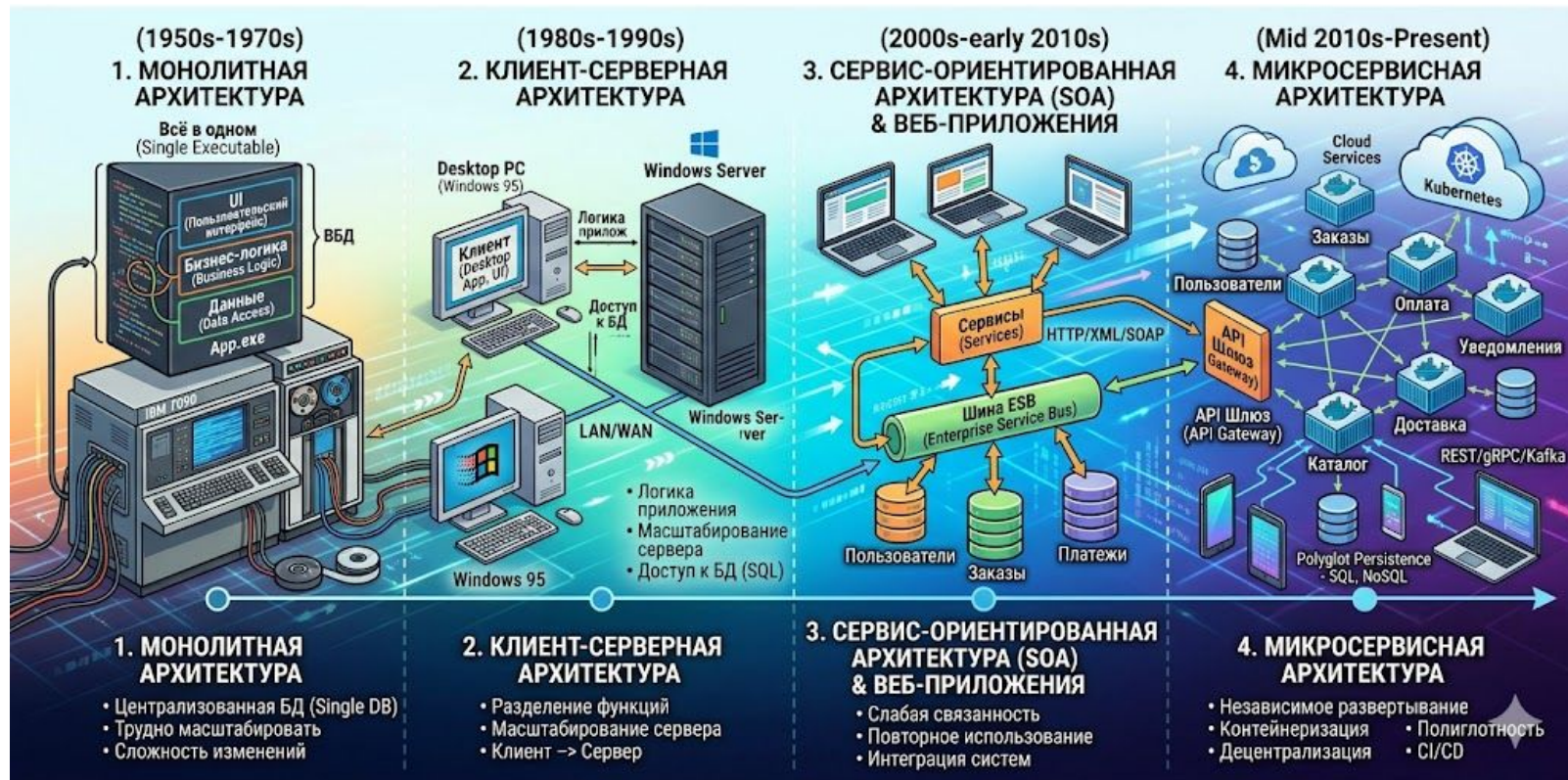
- Понимаем контекст на базе требований
- Определяем структуру компонентов
- Принимаем архитектурные решения, управляем компромиссами
- Выбираем технологии и инструменты
- Учитываем пожелания участников архитектурного проектирования



Гайд по работе с архитектурой



Эволюция архитектурных стилей



Монолитные архитектуры

Критерии

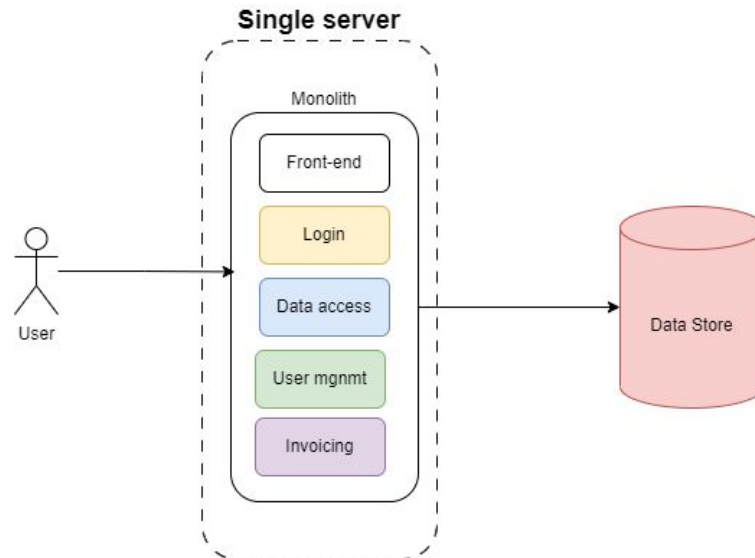
- Единая точка разработки и деплоя
- Единая база данных
- Единый цикл релиза для всех изменений
- В одном компоненте реализовано несколько бизнес-задач

Плюсы

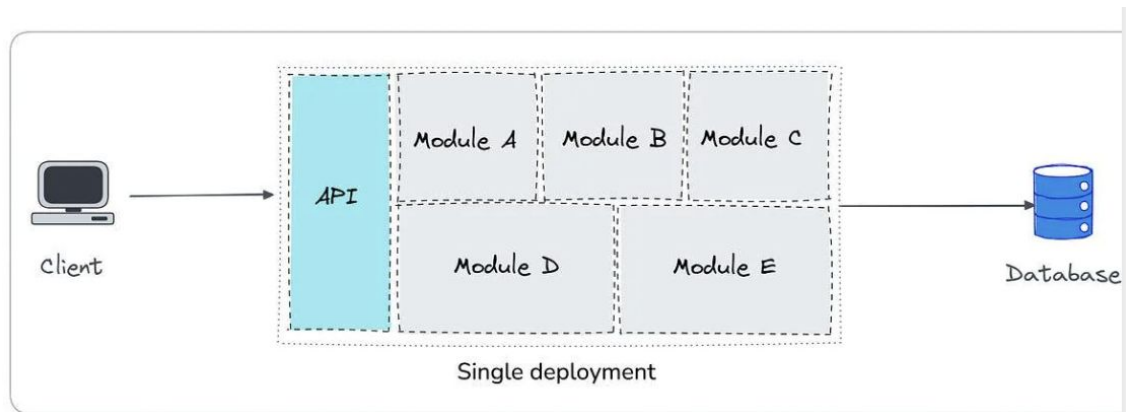
- Легко разрабатывать и отлаживать
- Между компонентами не участвует сеть, все быстро
- Простой деплой, т.к. один процесс
- Простое E2E тестирование

Минусы

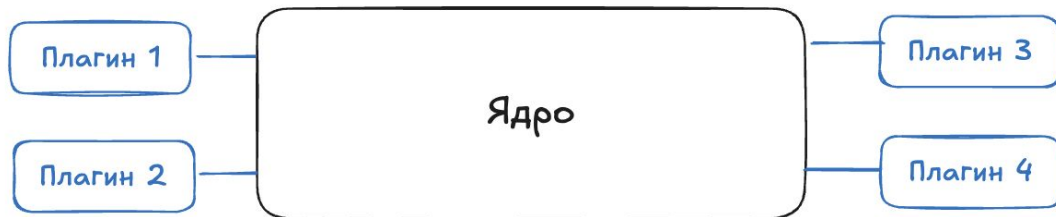
- Сложности с масштабированием компонентов
- Рост сложности при расширении команд, системы
- Сборка и деплой хоть и простые, но страшные
- Один технологический стек



Виды монолита



Модульный монолит

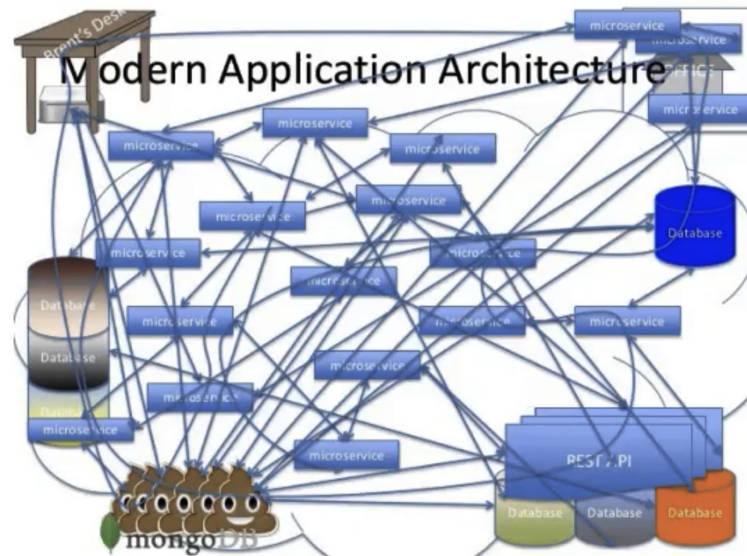


Микроядерный монолит

Распределенный монолит

Признаки

- Высокая связанность между сервисами, требующие координации при деплое
- Глобальные транзакции, которые охватывают несколько сервисов
- Отсутствие автономности сервисов
- Общие схемы данных. Использование общей базы данных или общей схемы между сервисами



Идея

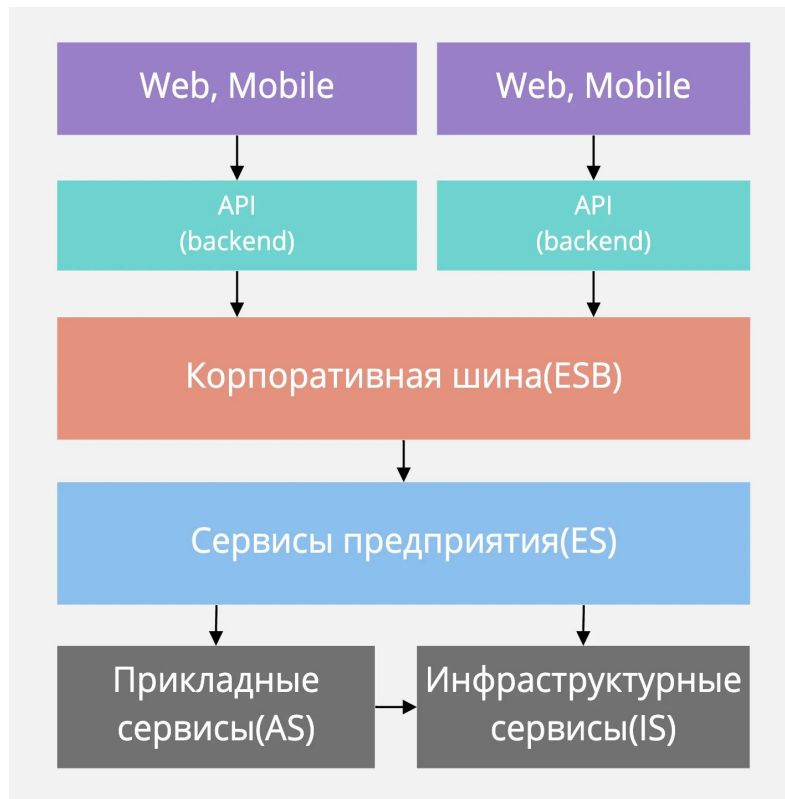
- Разбиваем монолит на сервисы, они общаются через стандартные протоколы
- Между сервисами ставим Enterprise Service Bus - централизованную шину для маршрутизации взаимодействий
- Принцип: loose coupling через стандартизированные контракты

Плюсы

- Переиспользование сервисов
- Четкие контракты между командами
- Теоретически независимый деплой сервисов

Минусы

- ESB - узкое место и точка отказа (умная шина, глупые сервисы)
- На практике SOA скатывался в распределенный монолит





Микросервисы

Определение

- Архитектурный стиль микросервисов - подход, при котором единое приложение строится как набор небольших сервисов, каждый из которых **работает в собственном процессе и коммуницирует** с остальными используя легковесные механизмы, как правило HTTP
- Эти сервисы построены вокруг бизнес-потребностей и **развертываются независимо с использованием полностью автоматизированной среды**. Сами по себе сервисы могут быть написаны **на разных ЯП** и использовать разные технологии хранения данных

Table 1 Relationship among microservices lifecycle stages, principles, features, and tools

Stage	Principle	Example features	Tools/practices
Design	Modeled around business domain	Contract, business, domain, functional, interfaces, bounded context, domain-driven design, single responsibility	Domain-driven design (DDD), bounded context
Design	Hide implementation details	Bounded contexts, REST, RESTful, hide databases, data pumps, event data pumps, technology-agnostic	OpenAPI, Swagger, Kafka, RabbitMQ, Spring Cloud Data Flow
Dev	Culture of automation	Automated, automatic, continuous*(deployment, integration, delivery), environment definitions, custom images, immutable servers	Travis-CI, Chef, Ansible, CI/CD
Dev	Decentralize all	DevOps, Governance, self-service, choreography, smart endpoints, dumb pipes, database-per-service, service discovery	Zookeeper, Netflix Conductor
Dev/ Ops	Isolate failure	Design for failure, failure patterns, circuit-breaker, bulkhead, timeouts, availability, consistency, antifragility,	Hystrix, Simian Army, Chaos Monkey
Ops	Deploy independently	versioning, one-service-per-host, containers	Docker, Kubernetes, canary A/B blue/green testing
Ops	Highly observable	Monitoring, logging, analytics, statistics, aggregation	ELK, Elasticsearch, Logstash, Kibana

Микросервисы

Идея

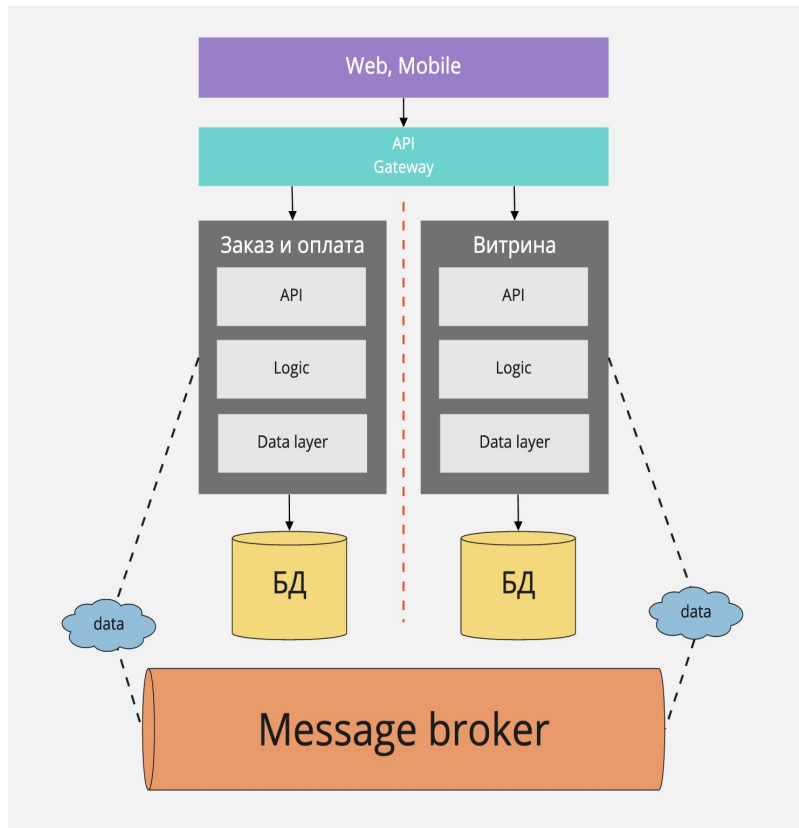
- Небольшие сервисы с принципами loose coupling and high cohesion, единая ответственность у сервиса и четкие границы
- Разный технологический стек внутри сервисов
- Независимый деплой, своя БД у каждого МС
- Легкие интеграции без ESB

Плюсы

- Независимость сборки и деплоя, самодостаточность
- Независимое масштабирование, изоляция отказов
- Технологическая свобода, ускорение time-to-market

Минусы

- Сложности распределенных систем (транзакции, консистентность)
- Операционная сложность (наблюдаемость, оркестрация)
- Много overhead: как в ресурсах, так и в процессах (QA)





Что обсудим дальше?

Часть №2

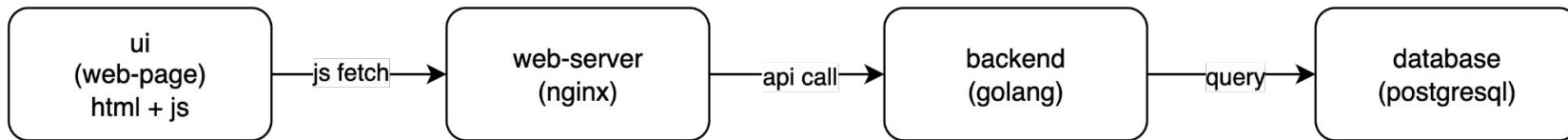
- Модульность, связанность, Domain Driven Design
- Связь “Архитектура” и “Фаза продукта”
- Связь “Архитектура” и “Инфраструктура”
- Другие архитектурные стили: Event-Driven Architecture, CQRS, паттерны Saga (оркестрация, хореография)
- Ренессанс монолитной архитектуры
- Будущее архитектуры
- Как выбрать архитектурный стиль?
- ATAM
- Cloud, Cloud Native
- Фреймворки описания архитектуры: UML, C4



Проектируем

Слои

1. backend - логика, в целом, достаточно
2. где хранить данные? приходит db
3. где показывать клиенту? приходит ui
4. а как показать клиенту? приходит web server

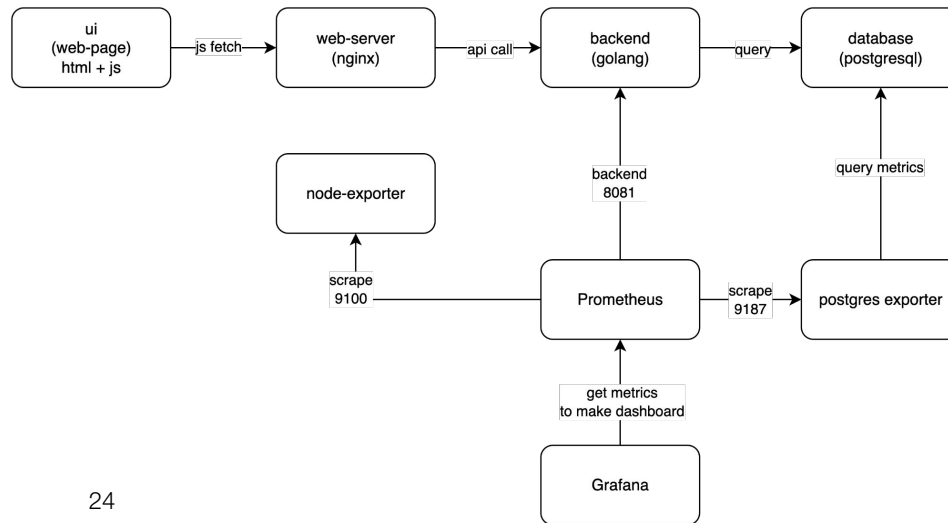




Проектируем

Слои

1. backend - логика, в целом, достаточно
2. где хранить данные? приходит db
3. где показывать клиенту? приходит ui
4. а как показать клиенту? приходит web server
5. а как понять, что все ок? приходят инструменты наблюдаемости





Домашнее задание

Будет выдано в .md



НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ